1. Briefly describe the artifact. What is it? When was it created?

The artifact selected for the enhancement was originally developed in CS300: DSA: Analysis and Design. The purpose of this C++ program is to read course information from a CSV file, validate prerequisite course numbers, store the course information in a binary search tree (BST) and allow users to display courses or search for an individual course and its prerequisites. The artifact uses a Course structure to store each course's number, name and prerequisite list. A node structure stores each Course and maintains pointers to left and right child nodes. The BinarySearchTree class provides recursive insertion, in-order traversal and search operations. The course number is used as the key for organizing the tree.

2. Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in algorithms and data structure? How was the artifact improved?

The original BST provides an average-case O(log n) insertion and search performance; however, its performance depends on the order in which courses are inserted. The tree can potentially become highly unbalanced, causing insertion and search to degrade to O(n) in the worst case and essentially behave like a linked list. This provided an opportunity to enhance the artifact with a self-balancing tree and improve its worst-case performance. The original BST was modified into a self-balancing AVL tree that maintains the height and balance factor of each node and uses rotations to keep the tree balanced. The AVL tree handles four possible imbalance cases: left-left (LL), right-right (RR), left-right (LR) and right-left (RL). The LL and RR cases are corrected with a single right or left rotation, respectively, while the LR and RL cases require two rotations to restore balance. This guarantees O(log n) worst-case insertion and search performance. I also compared the heights and performance of the original BST and the AVL tree

using mock data to demonstrate the differences between the two data structures. Additionally, I changed the prerequisite validation to utilize an unordered set, allowing course-number lookups to be average-case O(1) time instead of performing a linear O(n) search through the entire course catalog. These enhancements demonstrate my ability to analyze time complexity and select appropriate data structures and algorithms based on the problem being solved. They also show the importance of considering scalability and how a data structure's performance can become increasingly important as the amount of data grows, such as in an application containing hundreds of thousands or millions of elements.

3. Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome-coverage plans?

Yes I met the following outcome:

"Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution while managing the trade-offs involved in design choices."

This enhancement directly demonstrates this outcome because I analyzed a limitation of the original data structure, selected an AVL tree as a solution, implemented the necessary balancing algorithms and evaluated the results against the original BST. I also had to consider the trade-off between the additional complexity of implementing and maintaining an AVL tree and the benefit of guaranteed O(log n) worst-case insertion and search performance. The use of an unordered set for prerequisite validation also further displays the application of algorithmic analysis when selecting data structures for specific operations. My outcome plan currently remains the same and the final enhancement will meet the remaining outcomes.

4. Reflect on the process of enhancing and modifying the artifact. What did you learn as you were creating it and improving it? What challenges did you face?

Implementing the AVL tree itself was not difficult because it is an established data structure with many resources available explaining how it works. The main changes involved adding a height value to each node, creating functions to retrieve the height and calculate the balance factor, implementing left and right rotations and modifying the addNode function to check for the four possible AVL imbalance cases. Working through these changes helped me better understand how tree rotations can change the root of a subtree while preserving the ordering of the data.

I wanted to go beyond simply implementing the AVL tree and demonstrate its effect through actual performance comparisons. To do this, I needed a large amount of test data. Hand-writing thousands of records was not practical, so I initially used Mockaroo to generate mock data. However, its limitations led me to use the Python Faker library to create a script capable of generating 100,000 records. The generated data did not contain exactly the same type of course numbers and course names as the original application, but I determined that using the provided fields of passport numbers and names would still provide a valid way to compare tree height and search behavior. Prerequisites were retained in the smaller functional dataset because they have little effect on the tree's height or search performance.

I also modified the original BST implementation slightly so that I could measure tree height and search time and compare those results with the enhanced AVL implementation. In the 100,000 row test, the AVL tree produced a smaller tree, with a height of 20 compared to 38 for the original BST. However, the search-time results were not as dramatic as I initially expected. In some tests, the original BST was actually slightly faster than the AVL tree, and the differences

measured in nanoseconds were small. For example, the original BST achieved search times of 11,600 ns, 27,400 ns and 24,500 ns while the enhanced BST achieved search times of 16,300 ns, 25,400 ns and 25,600 ns. This showed me that theoretical complexity does not always translate into an obvious difference in a single small timing measurement. Factors such as the specific data distribution, system performance and the overhead introduced by maintaining a balanced tree can affect the observed runtime.

The most significant result occurred when I sorted the test data alphanumerically before inserting it into the trees. Sorted input is a realistic situation that could occur if course data were provided in an already ordered file. The AVL tree handled the sorted data without a significant change in its height, while the original BST was unable to complete the insertion because the tree became extremely unbalanced and the recursive calls eventually resulted in a stack overflow. I initially expected the original BST to simply produce a very tall tree, but I had not considered that the recursive implementation itself could fail before the tree could even be fully constructed. This was one of the most valuable lessons from the enhancement because it demonstrated a practical consequence of poor data-structure behavior that was not immediately obvious from simply looking at the Big-O complexity.

This enhancement taught me that evaluating an algorithm requires more than simply implementing it and comparing one timing measurement. The theoretical complexity, structure of the data, implementation details and characteristics of the test environment all affect the observed results. The AVL enhancement gave me a better understanding of how data structures behave under different conditions and reinforced the importance of testing both typical and worst-case scenarios.